RTOS DMAC
开发指南

版本号: 1.8

发布日期: 2025.09.20

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.7.9	AWA1636	初始版本
1.1	2022.07.30	XAA0190	增加 R128 描述
1.2	2023.04.23	XAA0190	1.3 节增加 MR527 描述
1.3	2023.08.21	XAA0190	1.3 节增加 XR875 描述
1.4	2024.03.11	XAA0228	1.3 节增加 AI985 描述
1.5	2024.10.11	AWA2138	1.1.3 节增加 V821 描述 2. 增加对 ndma 驱动的描述 3. 修改驱动框架，增加 ndma 的部分
1.6	2025.04.23	XAA0249	增加平台描述
1.7	2025.07.01	AWA2298	增加平台描述
1.8	2025.09.20	XAA0331	增加平台描述

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 2
1.4.1 标志说明 2
1.4.2 地址与数据描述方法约定 2
1.4.3 数值单位约定 3
1.5 相关术语介绍 3
1.5.1 硬件术语 3
1.5.2 软件术语 3

2 模块介绍 5
2.1 模块功能 5
2.2 模块配置 6
2.2.1 menuconfig 配置说明 6
2.2.2 使用配置说明 6
2.3 源码结构 7
2.4 软件框架 8

3 模块接口说明 9
3.1 数据结构 9
3.1.1 dma 传输配置结构体 10
3.2 hal_dma_chan_status_t hal_dma_chan_request 10
3.3 hal_dma_status_t hal_dma_chan_free 11
3.4 hal_dma_status_t hal_dma_chan_desc_free 11
3.5 hal_dma_status_t hal_dma_prep_cyclic 11
3.6 hal_dma_status_t hal_dma_prep_memcpy 12
3.7 hal_dma_status_t hal_dma_prep_device 12
3.8 hal_dma_status_t hal_dma_callback_install 13
3.9 hal_dma_status_t hal_dma_slave_config 13
3.10 enum dma_status hal_dma_tx_status 14
3.11 hal_dma_status_t hal_dma_start 14
3.12 hal_dma_status_t hal_dma_stop 14
3.13 void hal_dma_init(void) 15
3.14 void *dma_alloc_coherent 15
3.15 void dma_free_coherent 16

4 功能开发 17

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.1 功能概述 17
4.2 开发流程 17
4.3 编程示例 20

5 日志分析 22
5.1 调试信息 22
5.2 调试工具 23

6 FAQ 24
6.1 一号通 FAQ 列表 24
6.2 如何选择和配置 DMA 传输模式 24
6.2.1 两种模式原理 24
6.2.2 模式配置方法 26
6.2.3 wait 模式等待时间配置方法 27
6.2.4 wait 模式配置等待时间过短易出现的典型场景 27

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

介绍 RTOS 中 DMA 驱动的接口及使用方法，为 DMA 的使用者提供参考。

1.2 目标读者

- DMA 驱动开发人员
- DMA 驱动使用人员

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
F133	Melis	hal_dma.c
F135	Melis	hal_dma.c
V833	Melis	hal_dma.c
R328	FreeRTOS	hal_dma.c
R329-DSP	FreeRTOS	hal_dma.c
R128	FreeRTOS	hal_dma.c
MR527	FreeRTOS	hal_dma.c
XR875	FreeRTOS	hal_dma.c
AI985	FreeRTOS	hal_dma.c
V821	FreeRTOS	hal_ndma.c
MR153	FreeRTOS	hal_dma.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

产品名称	内核版本	驱动文件
T153	FreeRTOS	hal_dma.c
V861	FreeRTOS	hal_ndma.c

1.4 文档约定

1.4.1 标志说明

注意

提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000
	1 M	1 000 000
	1 G	1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
DMA	Direct Memory Access，直接存储器存取

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
HAL	Hardware Abstraction Layer，硬件抽象层

版权所有 © 珠海全志科技股份有限公司。保留一切权利

术语	解释说明
RTOS	Real Time Operatiing System，实时操作系统
GPIO	General Purpose Input/Output，通用输入输出
DRQSRC_XXX	源 DRQ 号
DRQDST_XXX	目的 DRQ 号

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能

DMA 主要实现设备与设备、设备与 memory、memory 与 memory 之间的数据搬运与传输。BSP DMA 驱动主要实现设备驱动的底层细节，并为上层提供一套标准的 API 接口以供使用。DMA 驱动中有 sgdma（下称 dma）和 ndma 两种，区别如下。

1、sgdma：支持链式传输，驱动源码对应 hal_dma.c。链式传输格式说明如下

- 传输一大串数据的时候，需要把数据分成 n 块，每一块配置好描述符，组成一个链表。将第一个块的描述符地址传到描述符寄存器中，就可以开始链式传输了
- 每个描述符长度为 6word，包含 configuration、source address、destination address、byte counter、parameter、link。
- configuration：包含 DRQ type、传输端地址计数模式、传输 block 长度等；
- source address：配置传输的源地址；
- destination address：配置传输的目的地址；
- byte counter：配置一个数据包的数据量大小，最大不超过 2^25-1byte;
- parameter：配置一些参数，比如 timeout 等。
- link：指向下一个描述符地址，当 link 等于 0xFFFFF800 时，DMAC 会传输完这个包就停止传输。

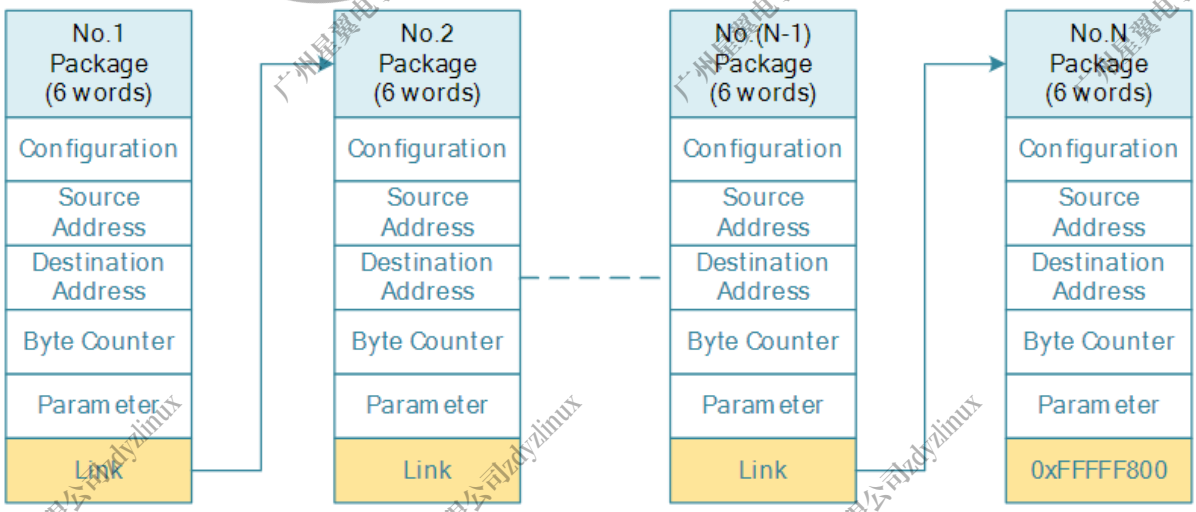

图 2-1: DMA 链式传输

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2、ndma：不支持链式传输，驱动源码对应 hal_ndma.c。传输一大段数据的时候，配置好源地址和目的地址，一次传输完，最大能传输 256Kbyte。

v821(sun300iw1) 的 rtos 驱动使用的是 ndma，其他用的是 sgdma。不管是 ndma 还是 sgdma，对外接口都是一致的。

2.2 模块配置

2.2.1 menuconfig 配置说明

Melis 版本：make menuconfig 配置路径:

Kernel Setup —> Drivers Setup —> SoC HAL Drivers —> DMA Devices

FreeRTOS 版本：mrtos_menuconfig 配置路径：

Drivers Options —> soc related device drivers —> DMA Devices

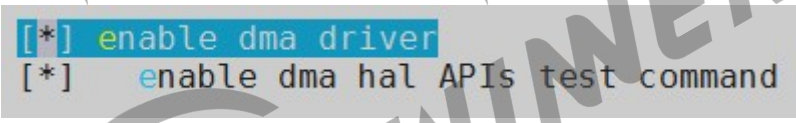

图 2-2: DMA menuconfig

处于 v821（sun300iw1）方案的时候，会默认选上 “enable ndma driver”，此时打开 ndma。

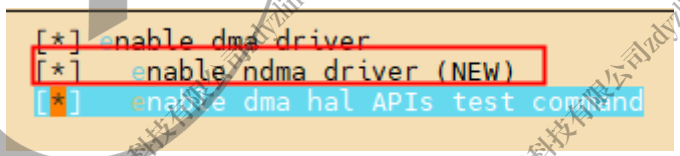

图 2-3: NDMA menuconfig

2.2.2 使用配置说明

在不同的 Sunxi 硬件平台中，DMA 设计不同，但平台配置文件的信息基本类似，一般位置为rtos-hal/hal/source/dma/platform/dma-xx.h。

内容如下：

```
#define SUNXI_DMAC_PBASE      0x03002000
#define DMA_IRQ_NUM           42

/*
* The source DRQ type and port corresponding relation
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
 */
#define DRQSRC_SRAM          0
#define DRQSRC_SDRAM         0
#define DRQSRC_DAUDIO_0_RX   3
#define DRQSRC_DAUDIO_1_RX   4
#define DRQSRC_AUDIO_CODEC   6
...
/*
 * The destination DRQ type and port corresponding relation
 */
#define DRQDST_SRAM          0
#define DRQDST_SDRAM         0
#define DRQDST_DAUDIO_0_TX   3
#define DRQDST_DAUDIO_1_TX   4
#define DRQDST_AUDIO_CODEC   6
...
```

其中：

1．SUNXI_DMAC_PBASE，表示 DMAC 基地址。

2．DMA_IRQ_NUM，表示 DMAC 中断号。

3．DRQSRC_XXX，表示源 DRQ 号。

4．DRQDST_XXX，表示目的 DRQ 号。

2.3 源码结构

```
hal/source/dma/          # 驱动源码
 ├──── hal_dma.c          # dma驱动源码
 ├──── hal_ndma.c         # ndma驱动源码
 ├──── Kconfig            # dma驱动的menuconfig配置文件
 ├──── Makefile           # dma驱动的makefile文件
 ├──── platfrom-nd        # ndma的芯片差异配置头文件
 │   ├──── ndma-sun300iw1.h
 ├──── platform           # dma的芯片差异配置头文件
 │   ├──── dma-sun8iw18.h
 │   └──── dma-sun8iw19.h
 │   └──── dma-sun8iw20.h
 ├──── platform-dma.h     # ndma的芯片差异配置总头文件
 ├──── platform-ndma.h    # dma的芯片差异配置总头文件

include/hal/             # 驱动APIs声明头文件
 └──── hal_dma.h

hal/test/dma/            # 驱动APIs测试代码
 ├──── Makefile
 └──── test_dma.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4 软件框架

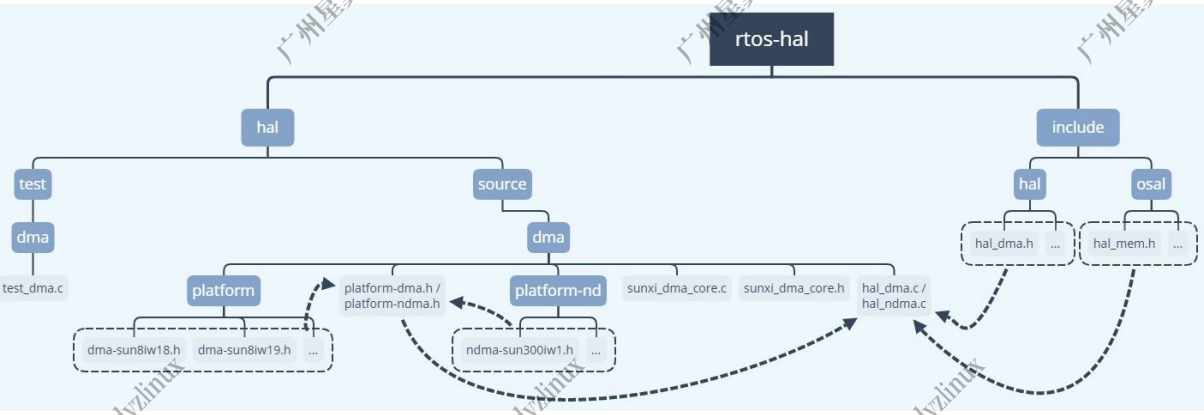

图 2-4: DMA 驱动框架

如上图是 dma 驱动的软件框架。主要包括 source，test，hal，osal 四部分。

- source

这是 dma 的驱动源码。其中 platform 和 platform-nd 文件里包含 dma 和 ndma 的不同芯片的寄存器信息，中断源等差异化配置。hal_dma.c 和 hal_ndma.c 分别是 dma 和 ndma 的驱动源码。

- hal

这是 dma 驱动 apis 的声明头文件。应用开发中可以包含这个头文件，即可使用 dma 的接口。dma 和 ndma 底层实现有差异，但是对外支持的接口一致的，通过这个头文件透出接口。

- osal

rtos 平台的公共 api，提供了 memry，delay，os 等相关的接口。

- test

test_dma.c 这是 dma 的测试示例。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

使用下述模块接口，需包含头文件：

```
#include <hal_dma.h>
```

API	解释说明
hal_dma_chan_request	申请 DMA 通道
hal_dma_chan_free	释放 DMA 通道
hal_dma_chan_desc_free	释放 DMA 通道描述符
hal_dma_prep_cyclic	初始化环形 DMA 传输
hal_dma_prep_memcpy	初始化 memory to memory DMA 传输
hal_dma_prep_device	初始化通用 DMA 传输
hal_dma_callback_install	注册 DMA 回调函数
hal_dma_slave_config	配置 DMA 描述符传输信息
hal_dma_tx_status	获取 DMA 发送状态
hal_dma_start	启动 DMA 传输
hal_dma_stop	停止 DMA 传输
hal_dma_init	初始化 DMA 控制器驱动
dma_alloc_coherent	申请一致性内存
dma_free_coherent	释放一致性内存

3.1 数据结构

在 dma 传输的时候，方向，数据类型等参数都需要配置。下面介绍配置这些数据的结构体。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.1.1 dma 传输配置结构体

```
struct dma_slave_config {
    enum dma_transfer_direction direction;  //数据传输方向
    enum dma_continue_mode conti_mode;     //连续模式
    unsigned long src_addr;            //数据源地址
    unsigned long dst_addr;            //数据目的地址
    enum dma_slave_buswidth src_addr_width; //源数据传输位宽
    enum dma_slave_buswidth dst_addr_width; //目的数据传输位宽
    uint32_t src_maxburst;            //源数据传输dma触发最小单元
    uint32_t dst_maxburst;            //目的数据传输dma触发最小单元
    uint32_t slave_id;                //dma id号
};
```

- direction

dma 支持内存（mem）和设备（dev）之间相互搬运，一共支持 4 中模式：DMA_MEM_TO_MEM，DMA_MEM_TO_DEV，DMA_DEV_TO_MEM，DMA_DEV_TO_DEV；

- conti_mode

v821 平台驱动新支持上的功能，一旦启动，dma 搬运一直运行。直到配置停止 dma loading。

- src_addr_width、dst_addr_width、src_maxburst、dst_maxburst

其中 burst 表示 dma 传输中最小触发单元。addr_width 表示每个单元的数据位宽。比如，burst=1，addr_wid=32bit，那么一次传输最小单元就是 1*32bit。要注意的是 src 和 dst 的 burst*addr_wid 要一致！

- slave_id

这是配置每个 dma 通道对应的 DRQ 号，可以查找一号通上 xx_User_Manual 文档的 dma 章节介绍的《DMA DRQ 类型》篇章。

3.2 hal_dma_chan_status_t hal_dma_chan_request

- 原　型：hal_dma_chan_status_t hal_dma_chan_request(struct sunxi_dma_chan **dma_chan)
- 作用：申请 DMA 通道
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - dma_chan: 存放 DMA 通道的指针变量

- 返回：
  - HAL_DMA_CHAN_STATUS_BUSY: 申请失败
  - HAL_DMA_CHAN_STATUS_FREE: 申请成功

3.3 hal_dma_status_t hal_dma_chan_free

- 原型：hal_dma_status_t hal_dma_chan_free(struct sunxi_dma_chan *chan)
- 作用：释放 DMA 通道
- 参数：
  - chan: 要释放的 DMA 通道结构体指针变量
- 返回：
  - HAL_DMA_STATUS_ERROR: 失败
  - HAL_DMA_STATUS_OK: 成功

3.4 hal_dma_status_t hal_dma_chan_desc_free

- 原型：hal_dma_status_t hal_dma_chan_desc_free(struct sunxi_dma_chan *chan)
- 作用：释放 DMA 通道描述符
- 参数：
  - chan: 要释放的 DMA 通道结构体指针变量
- 返回：
  - HAL_DMA_STATUS_ERROR: 失败
  - HAL_DMA_STATUS_OK: 成功

3.5 hal_dma_status_t hal_dma_prep_cyclic

- 原型：hal_dma_status_t hal_dma_prep_cyclic(struct sunxi_dma_chan *chan, uint32_t buf_addr, uint32_t buf_len, uint32_t period_len, enum dma_transfer_direction dir)
- 作用：初始化环形 DMA 传输

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 参数：
  - chan:DMA 通道结构体指针变量
  - buf_addr: 数据缓冲区
  - buf_len: 数据缓冲区长度
  - period_len: 单次 DMA 搬运长度
  - dir:DMA 传输方向
- 返回：
  - HAL_DMA_STATUS_INVALID_PARAMETER: 参数非法
  - HAL_DMA_STATUS_ERROR: 失败
  - HAL_DMA_STATUS_OK: 成功

说明

在 ndma 和 sgdma 驱动中都支持这个函数。不同的是，接口传输要传输的数据地址和长度的时候。在 sgdma 驱动中改函数会将数据分割成几块，并配置好描述符，进行链式传输。在 ndma 驱动中则是直接将数据全部传输，注意一次最大传输 256Kbyte。不过接口形式都是一致的。

3.6 hal_dma_status_t hal_dma_prep_memcpy

- 原型：hal_dma_status_t hal_dma_prep_memcpy(struct sunxi_dma_chan *chan, uint32_t dest, uint32_t src, uint32_t len)
- 作用：初始化 memory to memory DMA 传输
- 参数：
  - chan:DMA 通道结构体指针变量
  - dest: 目的地址
  - src: 源地址
  - len: 传输长度
- 返回：
  - HAL_DMA_STATUS_INVALID_PARAMETER: 参数非法
  - HAL_DMA_STATUS_ERROR: 失败
  - HAL_DMA_STATUS_OK: 成功

3.7 hal_dma_status_t hal_dma_prep_device

- 原型：hal_dma_status_t hal_dma_prep_device(struct sunxi_dma_chan *chan, uint32_t dest, uint32_t src, uint32_t len, enum dma_transfer_direction dir)

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 作用：初始化通用 DMA 传输
- 参数：
  - chan:DMA 通道结构体指针变量
  - dest: 目的地址
  - src: 源地址
  - len: 传输长度
  - dir:DMA 传输方向
- 返回：
  - HAL_DMA_STATUS_INVALID_PARAMETER: 参数非法
  - HAL_DMA_STATUS_ERROR: 失败
  - HAL_DMA_STATUS_OK: 成功

3.8 hal_dma_status_t hal_dma_callback_install

- 原 型：hal_dma_status_t hal_dma_callback_install(struct sunxi_dma_chan *chan, dma_callback callback, void *callback_param)
- 作用：注册 DMA 回调函数
- 参数：
  - chan:DMA 通道结构体指针变量
  - callback: 回调函数 handler
  - callback_param: 回调函数传参
- 返回：
  - HAL_DMA_STATUS_INVALID_PARAMETER: 参数非法
  - HAL_DMA_STATUS_OK: 成功

3.9 hal_dma_status_t hal_dma_slave_config

- 原 型：hal_dma_status_t hal_dma_slave_config(struct sunxi_dma_chan *chan, struct dma_slave_config *config)
- 作用：配置 DMA 描述符传输信息
- 参数：
  - chan:DMA 通道结构体指针变量
  - config:DMA 描述符结构体指针变量

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回：
  - HAL_DMA_STATUS_INVALID_PARAMETER: 参数非法
  - HAL_DMA_STATUS_OK: 成功

3.10 enum dma_status hal_dma_tx_status

- 原型：enum dma_status hal_dma_tx_status(struct sunxi_dma_chan *chan, uint32_t *left_size)
- 作用：获取 DMA 发送状态
- 参数：
  - chan:DMA 通道结构体指针变量
  - left_size: 存放剩余长度的指针变量
- 返回：
  - DMA_INVALID_PARAMETER: 参数非法
  - DMA_IN_PROGRESS: 正在进行
  - DMA_COMPLETE: 传输完成

3.11 hal_dma_status_t hal_dma_start

- 原型：hal_dma_status_t hal_dma_start(struct sunxi_dma_chan *chan)
- 作用：启动 DMA 传输
- 参数：
  - chan:DMA 通道结构体指针变量
- 返回：
  - HAL_DMA_STATUS_INVALID_PARAMETER: 参数非法
  - HAL_DMA_STATUS_ERROR: 失败
  - HAL_DMA_STATUS_OK: 成功

3.12 hal_dma_status_t hal_dma_stop

- 原型：hal_dma_status_t hal_dma_stop(struct sunxi_dma_chan *chan)

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 作用：停止 DMA 传输
- 参数：
  - chan:DMA 通道结构体指针变量
- 返回：
  - HAL_DMA_STATUS_INVALID_PARAMETER: 参数非法
  - HAL_DMA_STATUS_ERROR: 失败
  - HAL_DMA_STATUS_OK: 成功

3.13 void hal_dma_init(void)

- 原型：void hal_dma_init(void)
- 作用：初始化 DMA 控制器驱动
- 参数：
  - void
- 返回：
  - void

3.14 void *dma_alloc_coherent

- 原型：void *dma_alloc_coherent(size_t size)
- 作用：申请一致性内存
- 参数：
  - 参数 1: 申请内存的大小
- 返回：
  - ptr: 内存缓冲区指针
- Note: 需包含头文件sunxi_hal_common.h

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.15 void dma_free_coherent

- 原型：void dma_free_coherent(void *addr)
- 作用：释放一致性内存
- 参数：
  - addr: 内存缓冲区指针
- 返回：
  - void
- Note: 需包含头文件sunxi_hal_common.h

广州星翼电子科技有限公司zdyzlinux

4 功能开发

4.1 功能概述

DMA 驱动提供了以下常用功能：

- dma 四种传输方式（mem2mem、mem2dev、dev2mem、dev2dev）
- 链式和非链式传输（v821 rtos 平台使用 ndma 驱动，不支持链式传输，但是使用接口是一致的）

4.2 开发流程

步骤 1 刷 cache

如果打开 cache 的情况下，需要使用 clean cache 接口。将对应的数据清出 cache，即把数据写回到 RAM 中。触发写回。解决 DMA 发送时 RAM 中的 buffer 数组没更新的问题。

刷 cache 的函数只是针对 mem 中的数据。当 mem2dev 和 dev2mem 的时候，只需要处理 mem 的数据就好。

```
hal_dcache_clean_invalidate((unsigned long)buf1, DMA_TEST_LEN);
hal_dcache_clean_invalidate((unsigned long)buf2, DMA_TEST_LEN);
```

步骤 2 申请 dma 通道

不同芯片平台 dma 通道数不一样。比如 r128 有 16 个通道，v821 有 8 个。具体可参考一号通上 xx_User_Manual 文档的 dma 章节。下面接口是在 dma 传输之前申请一个通道。通道是从 0 开始申请。申请了之后需要释放了，别的设备才可以用。

```
struct sunxi_dma_chan *hdma = NULL; //申明通道结构体变量

/* request dma chan */
  ret = hal_dma_chan_request(&hdma);
  if (ret == HAL_DMA_CHAN_STATUS_BUSY) {
    hal_log_err("dma channel busy!");
    goto end;
  }
```

步骤 3 注册 dma 回调函数

版权所有 © 珠海全志科技股份有限公司。保留一切权利

dma 中断触发方式主要有两种：全传输结束起中断，传输完一半起中断。注意的是当数据量比较小的时候，dma 搬运很快的，如果配置了半包起中断，传输到一半的时候可能没有中断起来，传输结束会来中断。

```
static void dma_test_cb(void *param)
{
}
/* register dma callback */
  ret = hal_dma_callback_install(hdma, dma_test_cb, hdma);
  if (ret != HAL_DMA_STATUS_OK) {
    hal_log_err("register dma callback failed!");
    goto end;
  }
```

步骤 4 配置 dma 传输结构体

配置 dma 传输的结构体，其中 direction 根据场景需求配置（mem2dev，dev2mem，mem2dev，dev2dev）；

```
  struct dma_slave_config config = {0};
  config.direction = DMA_MEM_TO_MEM;
  config.dst_addr_width = DMA_SLAVE_BUSWIDTH_8_BYTES;
  config.src_addr_width = DMA_SLAVE_BUSWIDTH_8_BYTES;
  config.dst_maxburst = DMA_SLAVE_BURST_16;
  config.src_maxburst = DMA_SLAVE_BURST_16;
  if (test_dma_para_t.conti_mode)
    config.conti_mode = DMA_CONTI_ENABLE;
  config.slave_id = sunxi_slave_id(DRQDST_SDRAM, DRQSRC_SDRAM);

  ret = hal_dma_slave_config(hdma, &config);

  if (ret != HAL_DMA_STATUS_OK) {
    hal_log_err("dma config error, ret:%d", ret);
    goto end;
  }
```

如果是涉及到 dev 传输（dev2mem，mem2dev，dev2dev），那么对应的目的地或源地址配置成设备的寄存器地址。举个例子：twi 的接收，方向是 dev2mem，就是 twi 获取数据的寄存器里的数据搬运到内存（mem）。那么 src 地址就是 twi 设备的 rx 寄存器，dst 就是 mem 地址。

```
  slave_config.src_addr = twi->base_addr + TWI_DRIVER_RECVF;
  slave_config.dst_addr = (unsigned long)buf;
```

步骤 5 配置 dma 传输地址

这一步有三种接口可以选择

1.hal_dma_prep_memcpy

这是 memtomem 接口，使用如下：

```
ret = hal_dma_prep_memcpy(hdma, (unsigned long)buf2, (unsigned long)buf1, DMA_TEST_LEN);
  if (ret != HAL_DMA_STATUS_OK) {
    hal_log_err("dma prep error, ret:%d", ret);
    goto end;
  }
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2，hal_dma_prep_device

这个接口是针对 dev 的搬运的。相比 hal_dma_prep_memcpy 多了个配置方向的传参，支持 dev2dev，mem2dev，dev2mem。

```
hal_dma_prep_device(dma_chan, slave_config.dst_addr, slave_config.src_addr, len, DMA_MEM_TO_DEV);
```

3，hal_dma_prep_cyclic

这是支持链式传输的接口, 支持 mem2mem，dev2dev，mem2dev，dev2mem。这个接口有个传参（实例中的 uart_priv->uart_config->uart_rx_trigger_size），配置一次触发的长度。比如总传输 4096B，uart_rx_trigger_size 配置成 1024. 那么这些数据被分成 4 份，链式传输，每一份 1024B.

```
ret = hal_dma_prep_cyclic(uart_priv->rx_dma, uart_priv->rx_slave_config.dst_addr,
        uart_priv->uart_config->rx_buf_len,
        uart_priv->uart_config->uart_rx_trigger_size,
        uart_priv->rx_slave_config.direction);
```

说明

v821 不支持链式传输，但是这个接口也可以用。只是配置 trigger_size 没用，对于 v821 会一次性把总的数据全部传输完，一次传输的最大数是 256kB。

步骤 6 开始 dma 传输

```
hdma->irq_type = (test_dma_para_t.irq_type ? IRQ_PKG : IRQ_HALF);//配置中断起来方式：半包或全包
  ret = hal_dma_start(hdma);
  if (ret != HAL_DMA_STATUS_OK) {
    hal_log_err("dma start error, ret:%d", ret);
    goto end;
  }
```

步骤 7 等待 dma 传输结束

```
while (hal_dma_tx_status(hdma, &size)!= 0)
  {
    printf("wait dma transfer finash ... \n");
  }
```

步骤 8 停止 dma 传输和释放 dma 通道

dma 传输结束，停止 dma，释放通道。

```
ret = hal_dma_stop(hdma);
  if (ret != HAL_DMA_STATUS_OK) {
    hal_log_err("dma stop error, ret:%d", ret);
    goto end;
  }

  ret = hal_dma_chan_free(hdma);
  if (ret != HAL_DMA_STATUS_OK) {
    hal_log_err("dma free error, ret:%d", ret);
    goto end;
  }
```

步骤 9 无效 cache

版权所有 © 珠海全志科技股份有限公司。保留一切权利

使缓存中的数据为无效，cpu 下次使用该数据时，就会从 RAM 中重新载入。

```
hal_dcache_invalidate((unsigned long)buf2, DMA_TEST_LEN);
```

4.3 编程示例

1. 驱动测试示例

路径：lichee/rtos-hal/hal/test/dma/test_dma.c

```
int cmd_test_dma(int argc, char **argv)
{
    int ret, i;
    struct dma_slave_config config = {0};//声明dma配置结构体
    struct sunxi_dma_chan *hdma = NULL; //声明通道
    //申请两个cache line对齐的buf
    buf2 = hal_malloc_coherent(DMA_TEST_LEN);
    buf1 = hal_malloc_coherent(DMA_TEST_LEN);

    if (buf1 == NULL) {
        hal_log_err("malloc buf1 error!");
        goto end;
    }

    if (buf2 == NULL) {
        hal_log_err("malloc buf2 error!");
        goto end;
    }

    memset(buf1, 0, DMA_TEST_LEN);
    memset(buf2, 0, DMA_TEST_LEN);

    for (i = 0;i < DMA_TEST_LEN; i++)
        buf1[i] = i & 0xff;

    // clean cache
    hal_dcache_clean_invalidate((unsigned long)buf1, DMA_TEST_LEN);
    hal_dcache_clean_invalidate((unsigned long)buf2, DMA_TEST_LEN);

    /* request dma chan */
    ret = hal_dma_chan_request(&hdma);
    if (ret == HAL_DMA_CHAN_STATUS_BUSY) {
        hal_log_err("dma channel busy!");
        goto end;
    }

    /* register dma callback */
    ret = hal_dma_callback_install(hdma, dma_test_cb, hdma);
    if (ret != HAL_DMA_STATUS_OK) {
        hal_log_err("register dma callback failed!");
        goto end;

    config.direction = DMA_MEM_TO_MEM;
    config.dst_addr_width = DMA_SLAVE_BUSWIDTH_8_BYTES;
    config.src_addr_width = DMA_SLAVE_BUSWIDTH_8_BYTES;
    config.dst_maxburst = DMA_SLAVE_BURST_16;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    config.src_maxburst = DMA_SLAVE_BURST_16;
    config.slave_id = sunxi_slave_id(DRQDST_SDRAM, DRQSRC_SDRAM);

    ret = hal_dma_slave_config(hdma, &config);

    if (ret != HAL_DMA_STATUS_OK) {
        hal_log_err("dma config error, ret:%d", ret);
        goto end;
    }

    ret = hal_dma_prep_memcpy(hdma, (unsigned long)buf2, (unsigned long)buf1, DMA_TEST_LEN);
    if (ret != HAL_DMA_STATUS_OK) {
        hal_log_err("dma prep error, ret:%d", ret);
        goto end;
    }

    ret = hal_dma_start(hdma);
    if (ret != HAL_DMA_STATUS_OK) {
        hal_log_err("dma start error, ret:%d", ret);
        goto end;
    }

    hal_sleep(1);

    while (hal_dma_tx_status(hdma, &size)!= 0)
    {
        printf("wait dma transfer finash ... \n");
    }

    ret = hal_dma_stop(hdma);
    if (ret != HAL_DMA_STATUS_OK) {
        hal_log_err("dma stop error, ret:%d", ret);
        goto end;
    }

    ret = hal_dma_chan_free(hdma);
    if (ret != HAL_DMA_STATUS_OK) {
        hal_log_err("dma free error, ret:%d", ret);
        goto end;
    }

    hal_dcache_invalidate((unsigned long)buf2, DMA_TEST_LEN);
end:
    hal_free_coherent(buf1);
    hal_free_coherent(buf2);

    return 0;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 日志分析

5.1 调试信息

DMA 驱动默认输出 DEBUG 信息：

1. 打印 dma 驱动错误信息

dma 驱动路径：lichee/rtos/drivers/rtos-hal/hal/source/dma/hal_dma.c

ndma 驱动路径：lichee/rtos/drivers/rtos-hal/hal/source/ndma/hal_ndma.c

```
#define DMA_ERR(fmt, arg...) printf("%s()%d " fmt, __func__, __LINE__, ##arg)
```

2. 打印寄存器，dma 链配置等信息

路径：lichee/rtos/drivers/rtos-hal/include/hal/hal_dma.h

```
#define DMA_DEBUG
```

配置上述宏之后，会使用下面函数打印 dma 寄存器

```
static inline void sunxi_dump_chan_regs(u32 chan_num)
{
#ifdef DMA_DEBUG
   printf("Chan %d reg:\n"
        "\tDMA_IRQ_EN(0x%08lx):   0x%08x\n"
        "\tDMA_IRQ_STAT(0x%08lx): 0x%08x\n"
        "\tDMA_GATE(0x%08lx):     0x%08x\n"
        "\tDMA_GATE_LEN(0x%08lx): 0x%08x\n"
        "\tDMA_CFG(0x%08lx):      0x%08x\n"
        "\tDMA_CUR_SRC(0x%08lx):  0x%08x\n"
        "\tDMA_CUR_DST(0x%08lx):  0x%08x\n"
        "\tDMA_CNT(0x%08lx):      0x%08x\n\n",
        chan_num,
        DMA_IRQ_EN(chan_num), (uint32_t)hal_readl(DMA_IRQ_EN(chan_num)),
        DMA_IRQ_STAT(chan_num), (uint32_t)hal_readl(DMA_IRQ_STAT(chan_num)),
        DMA_GATE, (uint32_t)hal_readl(DMA_GATE),
        DMA_GATE_LEN, (uint32_t)hal_readl(DMA_GATE_LEN),
        DMA_CFG(chan_num), (uint32_t)hal_readl(DMA_CFG(chan_num)),
        DMA_CUR_SRC(chan_num), (uint32_t)hal_readl(DMA_CUR_SRC(chan_num)),
        DMA_CUR_DST(chan_num), (uint32_t)hal_readl(DMA_CUR_DST(chan_num)),
        DMA_CNT(chan_num), (uint32_t)hal_readl(DMA_CNT(chan_num)));
#endif
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2 调试工具

配置 CONFIG_HAL_TEST_DMA 后，可以使用 hal_dma 命令验证 DMA 模块是否能正常工作。

```
uart>hal_dma
DMA finished, callback to do something...
src buf:
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5

dst buf:
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5
a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5 a5

dma test pass
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 一号通 FAQ 列表

- FAQ2771

6.2 如何选择和配置 DMA 传输模式

DMA 传输模式分为 wait 模式和握手模式 (寄存器可配)，目前驱动默认使用 wait 模式，当使用 DMA 传输出现高负载下（如播放音频）fifo 会残留几个 byte 数据或者数据重复搬运等相似问题时，可以尝试通过改为握手模式或修改等待时间来解决类似问题。

6.2.1 两种模式原理

wait 模式:

1）当 DMA 检测到外部 request 信号有效，然后开始对设备进行操作，在达到 block 操作数目之前，内部 DRQ 一直保持高电平；

2）当 DMA 传输数量达到 block 操作数时，内部的 DRQ 自动拉低:

3）当内部 DRQ 自动拉低 wait counter 数量个 DMA 周期后，重新对外部的 request 信号进行检测，如果外部 request 信号有效，则开始下一轮传输

握手模式:

版权所有 © 珠海全志科技股份有限公司。保留一切权利

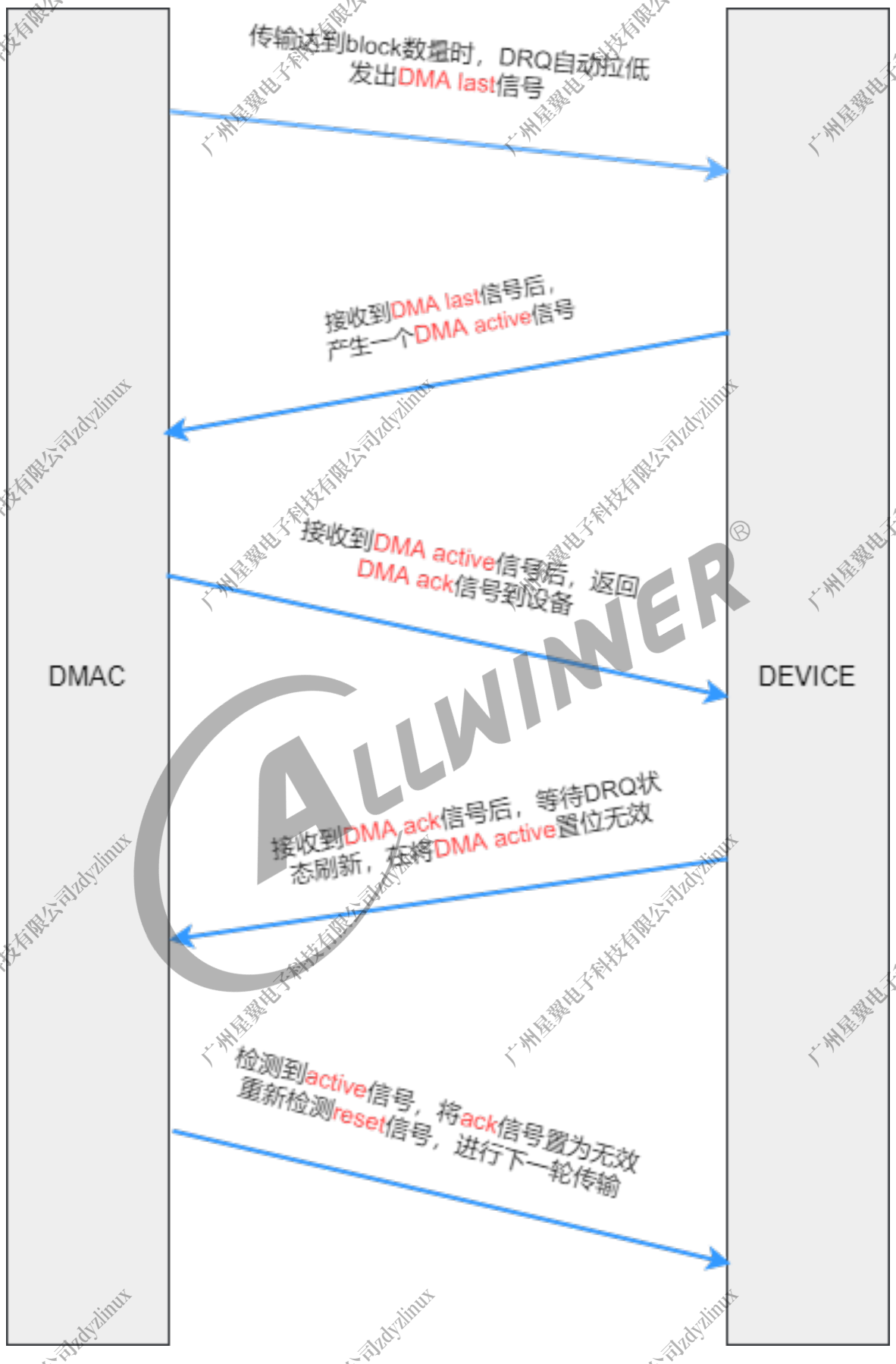

图 6-1: 握手模式示意图

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1）DMA 检测到外部 request 信号有效，然后开始对设备进行操作，在达到 block 操作数量之前，内部 DRQ 一直保持高电平;

2）当 DMA 传输数量达到 block 操作数时，内部的 DRQ 自动拉低，与此同时，在达到 block 数量的最后一次 DMA 操作中，DMA 会随操作命令同步发送 DMA last 信号。

3）DMA last 信号将被当做 DMA 命令的一部分在总线上传输，当设备端接收到总线上附加的 DMA last 操作命令后，可判定当前 DMA 传输 block 长度结束，即在重新发送下次请求之前不会再有 DMA 操作出现，设备然后产生一个 DMA active 信号到 DMA 控制器。需要注意的是，设备的每个 DRQ 信号对应一个 active 信号，如果设备具有多个 DRQ 则需根据不同的总线操作返回不同的 active 信号。

4）当 DMA 接收到设备端发送的 active 信号后，即返回 DMA ACK 信号到设备 (ACK 信号与 active 信号一对应)。

5）当设备端接收到 DMA ACK 信号后如果此时设备端所有线上操作已经完成，FIFO 状态和相应的 DRQ 状态已经刷新，则将 active 信号置为无效，否则等待 DRQ 状态刷新后再将 active 信号置无效。

6）当 DMA 检测到 active 信号的下降沿，相应的 ACK 信号会置为无效，并重新开始检测外部 request 信号，如果 request 信号有效，则开始下一轮传输。

二者区别：

1. 等待模式，基于时间等待（固定周期）搬运数据。在 DMA 搬运完一个 pkg 后，设备端 DRQ 仍然有效的话，DMA 等待 64 cycles 后（64 为默认值，可修改）继续搬运数据，若设备端数据在等待时间内还未完成更新的话，就会导致数据被重复搬运。为避免此问题，需根据实际场景配置合适的等待时间。

2. 握手模式，基于硬件信号交互搬运数据。在 DMA 搬运完一个 pkg 后，无论设备端 DRQ 是否有效，都会立即停止数据搬运，同时通过硬件应答信号（DMA last → active → ACK）确保双方确认数据传输完成后，再去检测 DRQ 仍有效时，才启动下一轮传输。这种应答机制确保数据不会重复搬运。该模式主要控制 DMA 传输启动条件，不影响单次数据传输流程。

如果使用场景对带宽和效率没有要求，优先使用握手模式，如果使用场景对带宽和效率有要求则优先使用 wait 模式，但是 wait 模式下等待时间需根据实际使用场景配置合适的等待时间，配置等待时间过短有可能造成数据错乱的情况。

6.2.2 模式配置方法

在目前 DMA 驱动代码中，默认使用 **wait 模式**，如果想要配置为握手模式，需调用’SET_OP_MODE‘函数进行模式配置

例如在 spi 中配置为握手模式方法如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

在spi代码hal_spi_master_status_t函数中配置SET_OP_MODE(sspi->dma_rx.chan->chan_count, SRC_HSMASK | DST_HS_MASK))

6.2.3 wait 模式等待时间配置方法

调用 sunxi_dma_set_wait_cycles 函数可以实现配置等待时间，该等待时间配置范围为 0~255 之间，单位为 cycles，**默认时间为 64 cycles**。

注意： 如果调用了 'hal_dma_prep_cyclic' 函数，要想配置等待时间生效，sunxi_dma_set_wait_cycles 的调用时机必须在该函数之后。

6.2.4 wait 模式配置等待时间过短易出现的典型场景

1. 高负载的情况（如播放音频）出现 fifo 会残留几个 byte 的情况，导致所有数据总体错位并且导致 DMA 无法搬运完 fifo 中的数据。
2. 数据重复搬运，一笔数据搬运两次

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。